

Chapter 5. Propensity Score

In [Chapter 4](#), you learned how to adjust for confounders using linear regression. In addition to that, you were introduced to the concept of debiasing through orthogonalization, which is one of the most useful bias-adjusting techniques available.

However, there is another technique that you need to learn—propensity weighting. This technique involves modeling the treatment assignment mechanism and using the model’s prediction to reweight the data, instead of building residuals like in orthogonalization. In this chapter, you will also learn how to combine the principles of [Chapter 4](#) with propensity weighting to achieve what is known as double robustness.

The content of this chapter is better suited for when you have binary or discrete treatments. Still, I’ll show an extension that allows you to use propensity weighting for continuous treatment.

The Impact of Management Training

A common phenomenon in tech companies is for talented individual contributors (ICs) to branch out to a management track. But because management often requires a very different skill set than the ones that made them talented ICs, this transition is often far from easy. It comes at a high

personal cost, not only for the new managers, but for those they manage.

Hoping to make this transition less painful, a big multinational company decided to invest in manager training for its new managers. Also, to measure the effectiveness of the training, the company tried to randomly select managers into this program. The idea was to compare an engagement score for the employees whose managers got enrolled in the program with the engagement of those whose managers didn't. With proper randomization, this simple comparison would give the average treatment effect of the training.

Unfortunately, things are not that simple. Some managers didn't want to go to the training, so they simply didn't show up. Others managed to get the training even without being assigned to receive it. The result is that what was to be a randomized study ended up looking a lot like an observational one.

NONCOMPLIANCE

People not getting the treatment they are intended to is called *noncompliance*. You will see more about this when we talk about instrumental variables in [Chapter 11](#).

Now, as an analyst who has to read this data, you'll have to make treated and untreated comparable by adjusting for confounders. To do that, you are given data on the company managers along with some covariates that describe them:

```
In [1]: import pandas as pd
import numpy as np

df = pd.read_csv("data/management_train.csv")
df.head()
```

	department_id	intervention	engagement_score
0	76	1	0.2773
1	76	1	-0.449
2	76	1	0.7697
3	76	1	-0.121
4	76	1	1.5261

The treatment variable is `intervention` and your outcome of interest is `engagement_score`, which is the average standardized engagement score for that manager's employees. Beyond the treatment and the outcome, the covariates in this data are:

`department_id`

A unique identifier for the department

`tenure`

The number of years the manager has been with the company (as an employee, not necessarily as a manager)

`n_of_reports`

The number of reports the manager has

`gender`

Categorical variable for manager identified gender

`role`

Job category inside the company

`department_size`

The number of employees in that same department

`department_score`

The average engagement score in that same department

`last_engagement_score`

The average engagement score for that manager in the previous iteration of the engagement survey

You hope that, by controlling for some or all of these variables, you might manage to reduce or even eliminate the bias when estimating the causal relationship between management training and employee engagement.

SIMULATED DATA

This dataset was adapted from the one used in the study “Estimating Treatment Effects with Causal Forests: An Application,” by Susan Athey and Stefan Wager.

Adjusting with Regression

Before moving on to propensity weighting, let's use regression to adjust for the confounders. In general, when learning something new, it's always a good idea to have some benchmark that you trust to compare to. Here the idea is to check if the propensity weighting estimate is at least inline with the regression one. Now, let's get to it.

For starters, if you simply compare treated and control groups, this is what you'll get:

```
In [2]: import statsmodels.formula.api as smf

        smf.ols("engagement_score ~ intervention",
                data=df).fit().summary().tables
```

	coef	std err	t	P> t
Intercept	-0.2347	0.014	-16.619	0.000
intervention	0.4346	0.019	22.616	0.000

But then again, this result is probably biased, since the treatment was not entirely random. To reduce this bias, you can adjust for the covariates you have in your data, estimating the following model:

$$engagement_i = \tau T_i + \theta X_i + e_i,$$

where X is all the confounders you have plus a constant column for the intercept. Additionally, gender and role are both categorical variables, so you have to wrap them in

`C()` inside your OLS formula:

```
In [3]: model = smf.ols("""engagement_score ~ intervention
+ tenure + last_engagement_score + department
+ n_of_reports + C(gender) + C(role)""").fit()

print("ATE:", model.params["intervention"])
print("95% CI:", model.conf_int().loc["intervention"])
```

```
Out[3]: ATE: 0.2677908576676856
        95% CI: [0.23357751 0.30200421]
```

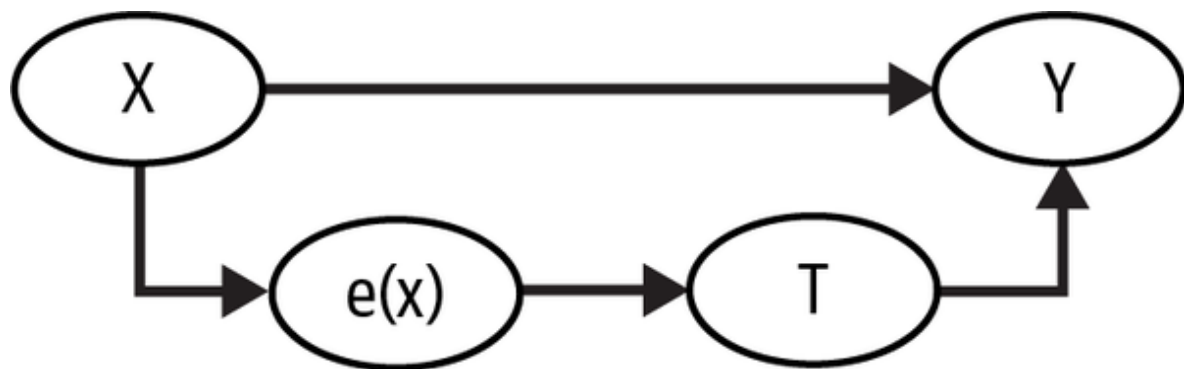
Notice that the effect estimate here is considerably smaller than the one you got earlier. This is some indication of positive bias, which means that managers whose employees were already more engaged are more likely to have participated in the manager training program. OK, enough with the preamble. Let's see what propensity weighting is all about.

Propensity Score

Propensity weighting revolves around the concept of a propensity score, which itself comes from the realization that you don't need to directly control for confounders X to achieve conditional independence $(Y_1, Y_0) \perp T | X$. Instead, it is sufficient to control for a balancing score that estimates $E[T | X]$. This balancing score is often the conditional probability of the treatment, $P(T | X)$, also called the *propensity score*, $e(x)$.

The propensity score can be viewed as a dimensionality reduction technique. Instead of conditioning on X , which can be very high dimensional, you can simply condition on the propensity score in order to block the backdoor paths that flow through X : $(Y_1, Y_0) \perp T | P(x)$.

There is [a formal proof](#) for why this is. It's not complicated, but a bit beyond the scope of this book. Here, you can approach the matter in a more intuitive way. The propensity score is the conditional probability of receiving the treatment, right? So you can think of it as some sort of function that converts X into the treatment T . The propensity score makes this middle ground between the variable X and the treatment T . This is what it would look like in a causal graph:



In this graph, if you know what $e(x)$ is, X alone gives you no further information about T . Which means that controlling for $e(x)$ works the same way as controlling for X directly.

Think of it in terms of the manager training program. Treated and nontreated are initially not comparable because the managers with more engaged direct reports are more likely to participate in the training. However, if you take two managers, one from the treated and one from the control group, but with the same probability of receiving the treatment, they are comparable. Think about it. If they have the exact same probability of receiving the treatment, the only reason one of them did it and the other didn't is by pure chance. Given the same propensity score, treatment is as good as random.

Propensity Score Estimation

In an ideal world, you would have the true propensity score $e(x)$. You might have this in the case of conditionally random experiment, where the assignment mechanism is

nondeterministic, but known. However, in most cases, the mechanism that assigns the treatment is unknown and you'll need to replace the true propensity score by an estimation of $e(x)$.

Since you have a binary treatment, a good candidate for estimating $e(x)$ is using logistic regression. To fit a logistic regression with `statsmodels`, you can simply change the method `ols` to `logit`:

```
In [4]: ps_model = smf.logit("""intervention ~
    tenure + last_engagement_score + departu
    + C(n_of_reports) + C(gender) + C(role)"""
```

Save your estimated propensity score in a data frame; you'll use it a lot in the following sections, where I'll show you how to use it and what it is doing:

```
In [5]: data_ps = df.assign(
    propensity_score = ps_model.predict
)

data_ps[["intervention", "engagement_score"]]
```

	intervention	engagement_score	pr
0	1	0.277359	0.5
1	1	−0.449646	0.3
2	1	0.769703	0.6
3	1	−0.121763	0.5
4	1	1.526147	0.6



PROPENSITY SCORE AND ML

Alternatively, you can use machine learning models to estimate the propensity score. But they require you to be more careful. First, you must ensure that your ML model outputs a calibrated probability prediction. Second, you need to use out-of-fold predictions to avoid bias due to overfitting. You can use `sklearn`'s calibration module for the first task and the `cross_val_predict` function, from the model selection module, for the latter.

Propensity Score and Orthogonalization

If you recall from the previous chapter, according to the FLW theorem, linear regression also does something very similar to estimating a propensity score. In the debiasing step, it estimates $E[T|X]$. So, very much like the propensity score

estimation, OLS is also modeling the treatment assignment mechanism. This means you can also use propensity score $\hat{e}(X)$ inside a linear regression in order to adjust for the confounders X :

```
In [6]: model = smf.ols("engagement_score ~ intervention",  
                        data=data_ps).fit()  
model.params["intervention"]
```

```
Out[6]: 0.26331267490277066
```

The estimated ATE you get with this approach is remarkably similar to the one you got earlier, fitting a linear model with the treatment and confounder X . This is not at all surprising, as both approaches are simply orthogonalizing the treatment. The only difference is that OLS uses linear regression to model T , while this propensity score estimate was obtained from a logistic regression.

Propensity Score Matching

Another popular approach to control for the propensity score is the matching estimator. This method searches for pairs of units with similar observable characteristics and compares

the outcomes of those who received the treatment to those who did not. If you have a background in data science, you can think of matching as a simple K-Nearest-Neighbors (KNN) algorithm, where $K=1$. To start, you fit a KNN model on the treated units, using the propensity score as the only feature, and use it to impute Y_1 for the control group. Next, you fit a KNN model on the untreated units and use it to impute Y_0 for the treated units. In both cases, the imputed value is simply the outcome of the matched unit, where the match is based on the propensity score:

```
In [7]: from sklearn.neighbors import KNeighborsRegressor

T = "intervention"
X = "propensity_score"
Y = "engagement_score"

treated = data_ps.query(f"{T}==1")
untreated = data_ps.query(f"{T}==0")

mt0 = KNeighborsRegressor(n_neighbors=1)

mt1 = KNeighborsRegressor(n_neighbors=1)

predicted = pd.concat([
    # find matches for the treated look
    treated.assign(match=mt0.predict(tr
```

```
# find matches for the untreated loc
untreated.assign(match=mt1.predict(1
])

predicted.head()
```

	department_id	intervention	engag
0	76	1	0.2773
1	76	1	-0.449
2	76	1	0.7697
3	76	1	-0.121
4	76	1	1.5261

Once you have a match for each unit, you can estimate the ATE:

$$\widehat{ATE} = \frac{1}{N} \sum \{(Y_i - Y_{jm}(i))T_i + (Y_{jm}(i) - Y_i)(1 - T_i)\},$$

where $Y_{jm}(i)$ is the match of unit i from the treatment group different from i 's:

```
In [8]: np.mean((predicted[Y] - predicted["match"]  
                + (predicted["match"] - predicted
```



```
Out[8]: 0.28777443474045966
```

BIAS OF THE MATCHING ESTIMATOR

You don't need to use matching only with the propensity score. Instead, you could directly match on the raw features X used to construct the propensity score estimate $\hat{P}(T|X)$. However, the matching estimator is biased, and that bias increases with the dimension of X . With a high dimensionality, the data becomes sparse, and the matching becomes poorer. Say $\mu_0(X)$ and $\mu_1(X)$ are the expected outcome function in the control and treated group, respectively. The bias is the discrepancy between the expected outcome and the match's outcome: $\mu_0(X_i) - \mu_0(X_{jm})$ for the treated units and $\mu_1(X_{jm}) - \mu_1(X_i)$ for the control units, where X_{jm} are the covariates for the match.

This means that, if you want to use matching, but avoid its bias, you have to apply a bias correction term:

$$\widehat{ATE} = \frac{1}{N} \sum \left\{ (Y_i - Y_{jm}(i) - (\hat{\mu}_0(X_i) - \hat{\mu}_0(X_{jm})))T_i + (Y_{jm}(i) - Y_i - (\hat{\mu}_1(X_{jm}) - \hat{\mu}_1(X_i)))(1 - T_i) \right\},$$

where $\hat{\mu}_1$ and $\hat{\mu}_0$ can be estimated with something like linear regression.

To be completely honest, I'm not a huge fan of this estimator, first, because it is biased, second, because it is difficult to derive its variance, and third, because my experience in data science has led me to be suspicious of KNN, mostly because it

is very inefficient with high dimensional X . This last problem is not an issue if you only match on the propensity score, but the first two problems remain. I'm teaching this method here mostly because it is very famous and you might see it here and there.

SEE ALSO

The paper “Why Propensity Scores Should Not Be Used for Matching,” by King and Nielsen, provides a more technical discussion on the issue with propensity score matching.

Inverse Propensity Weighting

There is another widely used approach for utilizing propensity scores that I find preferable—inverse propensity weighting (IPW). By reweighting the data based on the inverse probability of treatment, this method can make the treatment appear to have been randomly assigned in the reweighted data. To do this, we reweight the sample by $1/P(T = t|X)$ in order to create a pseudo-population that approximates what would have happened if everyone had received the treatment t :

$$E[Y_t] = E\left[\frac{\mathbb{1}(T = t)Y}{P(T = t|X)}\right]$$

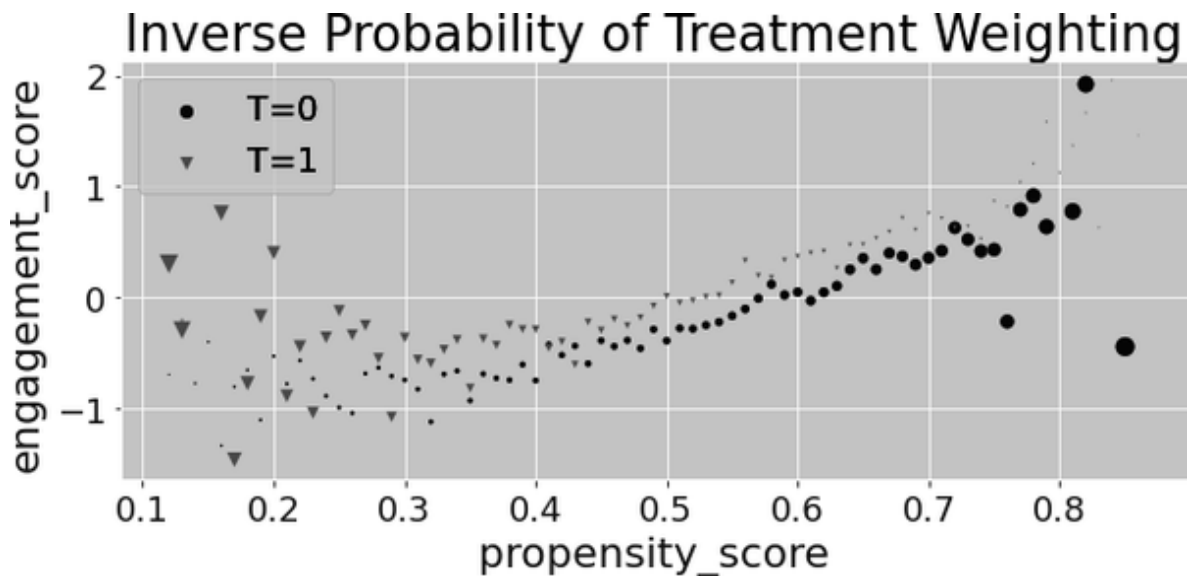
Once again, the proof for this is not complicated, but beside the point here. So let's stick with intuition. Suppose you want to know the expectation of Y_1 , that is, of what would be the

average engagement had all managers taken the training. To get that, you take all those who are treated and scale them by the inverse probability of getting the treatment. This puts a high weight on those with very low probability of treatment, but that nonetheless got it. You are essentially up-weighting rare treatment examples.

This makes sense, right? If a treated individual has a low probability of treatment, that individual looks a lot like the untreated. This must be interesting! This treated unit that looks like the untreated will probably be very informative of what would happen to the untreated, had they been treated, $Y_1|T = 0$. That is why you give a high weight to that unit.

Same thing for the control. If a control unit looks a lot like the treated group, it is probably a good estimate for $Y_0|T = 1$, so you give it more weight.

Here is what this process looks like with the management training data, with weights depicted as the size of each dot:



Notice how those managers who got the training, $T = 1$, have a high weight when $\hat{e}(X)$ is low. You are giving high importance to the treated that look like the untreated.

Conversely, the untreated have a high weight when $\hat{e}(X)$ is high, or when $\hat{P}(T = 0|X)$ is low. Here you are giving high importance to the untreated that look like the treated.

If you can use the propensity score to recover the average potential outcome, it also means you can use it to recover the ATE:

$$ATE = E\left[\frac{\mathbb{1}(T = 1)Y}{P(T = 1|X)}\right] - E\left[\frac{\mathbb{1}(T = 0)Y}{P(T = 0|X)}\right]$$

Both expectations can be estimated from data with very simple code:

```
In [9]: weight_t = 1/data_ps.query("intervention")
weight_nt = 1/(1-data_ps.query("intervention"))
```

```

t1 = data_ps.query("intervention==1")["e"]
t0 = data_ps.query("intervention==0")["e"]

y1 = sum(t1*weight_t)/len(data_ps)
y0 = sum(t0*weight_nt)/len(data_ps)

print("E[Y1]:", y1)
print("E[Y0]:", y0)
print("ATE", y1 - y0)

```

```

Out[9]: E[Y1]: 0.11656317232946772
        E[Y0]: -0.1494155364781444
        ATE 0.2659787088076121

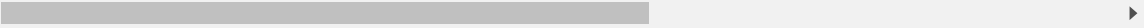
```

Using this approach, the ATE is once again smaller than the one you got naively, not adjusting for X . Moreover, this result looks pretty similar to the one you got by using OLS, which is a good check to make sure you didn't do anything wrong. It is also worth noticing that the ATE expression can be simplified to the following:

$$ATE = E\left[Y \frac{T - e(x)}{e(x)(1 - e(x))}\right]$$

Sure enough, it produces the exact same result as before:

```
In [10]: np.mean(data_ps["engagement_score"]  
                * (data_ps["intervention"] - data_ps["control"])  
                / (data_ps["propensity_score"] - data_ps["control"])
```



```
Out[10]: 0.26597870880761226
```

REGRESSION AND IPW

The preceding formula is very neat because it also gives you some insight into how IPW compares to regression. With regression, you are recovering the treatment effect with

$$\tau_{ols} = \frac{E[Y(T - E[T|X])]}{E[Var(T|X)]}.$$

With that in mind, recall that the variance of a Bernoulli variable with probability p is simply $p(1 - p)$. Hence, the IPW is recovering the treatment effect with

$$\tau_{ipw} = E\left[\frac{Y(T - E[T|X])}{Var(T|X)}\right].$$

Notice the similarity? To make it more transparent, since $1/E[Var(X|T)]$ is a constant, you can move it inside the expectation and rewrite the regression estimator as follows:

$$\tau_{ols} = E\left[\frac{Y(T - E[T|X])}{E[Var(T|X)]}\right] = E\left[\frac{Y(T - E[T|X])}{Var(T|X)} * W\right]$$

with $W = \text{Var}(T|X) / E[\text{Var}(T|X)]$.

Now, the thing inside the expectation in the IPW estimator identifies the effect (CATE) in the groups defined by X . So, the difference between IPW and OLS is the first weights each sample by 1, while regression weights the group effects by the conditional treatment variance. This is in line with what you learned in the previous chapter, about regression upweighting effects where the treatment varies a lot. So, even though regression and IPW look different, they are doing almost same thing, up to the weighting point.

Variance of IPW

Unfortunately, computing the standard error for IPW is not as straightforward as with linear regression. The most straightforward way to obtain a confidence interval around your IPW estimate is by using the bootstrap method. With this method, you will repeatedly resample the data with replacement to obtain multiple IPW estimators. You can then calculate the 2.5th and 97.5th percentiles of these estimates to obtain a 95% confidence interval.

To code that, let's first wrap your IPW estimation into a reusable function. Notice how I'm replacing

`statsmodels` with `sklearn`. The `logit` function in `statsmodels` is slower than the logistic regression model from `sklearn`, so this change will

save you some time. Also, since you probably don't want to lose the convenience of formulas you get from

`statsmodels`, I'm using `patsy`'s `dmatrix` function. This function engineers a feature matrix based on an R-style formula, like the ones you've been using so far:

```
In [11]: from sklearn.linear_model import LogisticRegression
         from patsy import dmatrix

         # define function that computes the IPW ATE
         def est_ate_with_ps(df, ps_formula, T, X_col):

             X = dmatrix(ps_formula, df)
             ps_model = LogisticRegression(penalty=None, max_iter=1000)
             ps = ps_model.predict_proba(X)[:, 1]

             # compute the ATE
             return np.mean((df[T]-ps) / (ps*(1-ps)))
```

PROBABILITY PREDICTION

By default, `sklearn`'s classifiers output 0 or 1 predictions following the logic $\hat{P}(Y|X) > 0.5$. Since you want your model to output a probability, you'll have to use the `predict_proba` method. This method outputs a two column matrix, where the first column is $\hat{P}(Y = 0|X)$ and the second column, $\hat{P}(Y = 1|X)$. You only want the second one, which in this case is $\hat{P}(T = 1|X)$. Hence, the indexing

`[:, 1]`.

Here is how you would use this function:

```
In [12]: formula = """tenure + last_engagement_score + C(n_of_reports) + C(gender) + C(role)
T = "intervention"
Y = "engagement_score"

est_ate_with_ps(df, formula, T, Y)
```

```
Out[12]: 0.2659755621752663
```

Now that you have the code to compute ATE inside a neat function, you can apply it inside a bootstrap procedure. To speed things up, I'm also going to run the resampling in parallel. All you have to do is call the data frame method `.sample(frac=1, replace=True)` to get a bootstrap sample. Then, pass this sample to the function you created earlier. To make the bootstrap code more generic, one of its arguments is an estimator function, `est_fn`, which takes a data frame and returns a single number as an estimate. I'm using four jobs, but feel free to set this to the number of cores in your computer.

Run this estimator multiple times, one in each bootstrap sample, and you'll end up with an array of estimates. Finally,

to get the 95% CI, just take the 2.5 and 97.5 percentiles of that array:

```
In [13]: from joblib import Parallel, delayed #

def bootstrap(data, est_fn, rounds=200, seed=1234):
    np.random.seed(seed)

    stats = Parallel(n_jobs=4) (
        delayed(est_fn) (data.sample(frac=0.5, replace=True,
                                      n=int(len(data) * 0.5))
        for _ in range(rounds)
    )

    return np.percentile(stats, pcts)
```

I tend to lean toward functional programming in my code, which might not be familiar to everyone. For this reason, I'll add notes explaining some of the functional patterns that I'm using, starting with the `partial` function.

PARTIAL

`partial` takes in a function and some of its arguments and returns another function just like the input one, but with the arguments that you passed already applied:

```
def addNumber(x, number):  
    return x + number  
  
add2 = partial(addNumber, number=2)  
add4 = partial(addNumber, number=4)  
  
add2(3)  
>>> 5  
  
add4(3)  
>>> 7
```

I'll use `partial` to take the `est_ate_with_ps` function and partially apply the formula, the treatment, and the outcome arguments. This will give me a function that has a data frame as its only input and that outputs the `$ATE$` estimate. I can then pass this function as the `est_fn` argument to the bootstrap function I created earlier:

```
In [14]: from toolz import partial  
  
         print(f"ATE: {est_ate_with_ps(df, form
```

```
est_fn = partial(est_ate_with_ps, ps_fn=ps_fn)
print(f"95% C.I.: ", bootstrap(df, est_fn))
```

```
Out[14]: ATE: 0.2659755621752663
          95% C.I.: [0.22654315 0.30072595]
```

This 95% is about as wide as the one you got earlier, with linear regression. It's important to realize that the variance in the propensity score estimator will be large if you have big weights. Big weights means that some units have a big impact in the final estimate. A few units having a big impact in the final estimate is precisely what causes the variance.

You'll have big weights if you have few control units in the region with high propensity score or a few treated units in the region with low propensity score. This will cause you to have few units to estimate the counterfactuals $Y_0|T = 1$ and $Y_1|T = 0$, which might give you a very noisy result.

CAUSAL CONTEXTUAL BANDITS

Contextual bandits is a flavor of reinforcement learning where the goal is to learn an optional decision-making policy. It merges a sampling component, which balances gathering data in unexplored regions with allocating the best treatment, and an estimation component, which tries to figure out the best treatment with the available data.

The estimation component can be easily framed as a causal inference problem, where you wish to learn the best treatment assignment mechanism, where *best* is defined in terms of the expected value of a desired outcome Y you wish to optimize. Since the algorithm goal is to allocate the treatment in an optimal manner, the data it gathers is confounded (not random). This is why a causal approach to contextual bandits can yield significant improvements.

If the decision-making process is probabilistic, you can store the probability of assigning each treatment, which is exactly the propensity score $e(x)$. Then, you can use this propensity score to reweight the past data, where the treatment has already been selected and the outcome is already observed. This reweighted data should be unconfounded and hence much easier to learn what is the optimal treatment.

Stabilized Propensity Weights

Weighting the treated samples by $1/P(T = 1|X)$ creates a pseudo-population the same size as the original one, but as though everyone was treated. This means that the sum of the weights is about the same as the original sample size.

Likewise, weighting the control by $1/P(T = 0|X)$ creates a pseudo-population that behaves as though everyone had the control.

If you come from a machine learning background, you might recognize IPW as an application of importance sampling. With importance sampling, you have data from an origin distribution $q(x)$ but want to sample from a target distribution $p(x)$. To do that, you can reweight the data from $q(x)$ by $p(x)/q(x)$. Bringing this to an IPW context, weighting the treated by $1/P(T = 1|X)$ essentially means you are taking data that came from $P(T = 1|X)$ —which is biased if X also causes Y —and reconstructing $P(T = 1) = 1$, where the treatment probability does not depend on X , since it is just 1. This also explains why the resulting re-weighted sample behaves as if everyone in the original sample was treated.

Another way to see that is to notice how the sum of the weights for both the treatment and the untreated are pretty close to the original sample size:

```
In [15]: print("Original Sample Size", data_ps.s
          print("Treated Pseudo-Population Sample
```

```
print("Untreated Pseudo-Population Sample Size")
```

```
Out[15]: Original Sample Size 10391  
         Treated Pseudo-Population Sample Size 10391  
         Untreated Pseudo-Population Sample Size 10391
```

This is fine, as long as you don't have weights that are too large. But if a treatment is very unlikely, $P(T|X)$ can be tiny, which might cause you some computational issues. A simple solution is to stabilize the weights using the marginal probability of treatment, $P(T = t)$:

$$w = \frac{P(T = t)}{P(T = t|X)}$$

With these weights, a treatment with low probability won't have massive weights because the small denominator will be balanced by the also small numerator. This won't change the results you got earlier, but it is more computationally stable.

Moreover, the stabilized weights reconstruct a pseudo-population where the effective size (sum of the weights) of both treated and control matches that of the original treated and control groups, respectively. Again, making a parallel with importance sampling, with stabilized weights, you are coming

from a distribution where the treatment depends on X , $P(T = t|X)$, but reconstructing the marginal $P(T = t)$:

```
In [16]: p_of_t = data_ps["intervention"].mean()

t1 = data_ps.query("intervention==1")
t0 = data_ps.query("intervention==0")

weight_t_stable = p_of_t/t1["propensity"]
weight_nt_stable = (1-p_of_t)/(1-t0["propensity"])

print("Treat size:", len(t1))
print("W treat", sum(weight_t_stable))

print("Control size:", len(t0))
print("W treat", sum(weight_nt_stable))
```

```
Out[16]: Treat size: 5611
W treat 5634.807508745978
Control size: 4780
W treat 4763.116999421415
```

Again, this stabilization keeps the same balancing properties of the original propensity score. You can verify that it yields the exact same ATE estimate as you had before:

```
In [17]: nt = len(t1)
         nc = len(t0)

         y1 = sum(t1["engagement_score"]*weight_
         y0 = sum(t0["engagement_score"]*weight_

         print("ATE: ", y1 - y0)
```

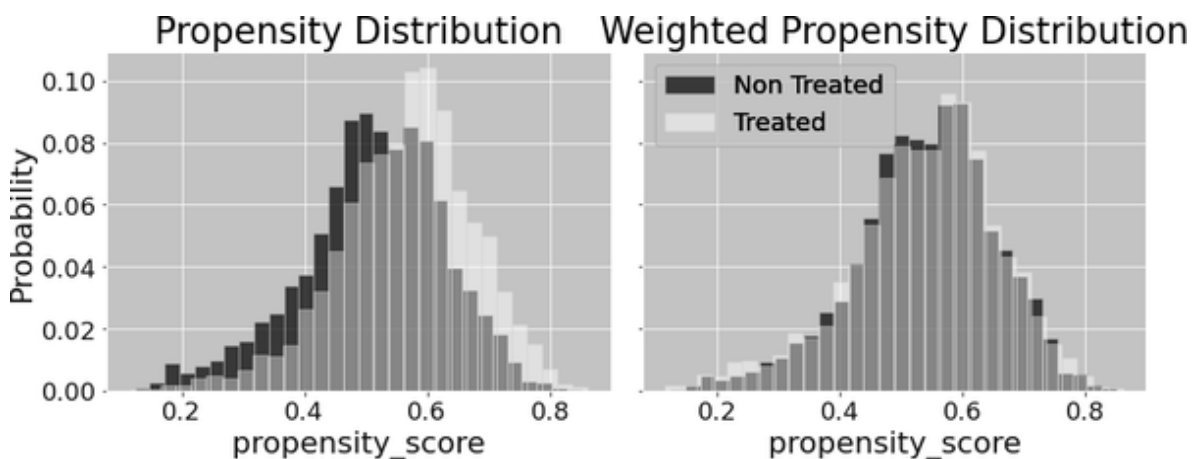
```
Out[17]: ATE:  0.26597870880761176
```

Pseudo-Populations

I've mentioned pseudo-populations already, but understanding them better will help you appreciate how IPW removes bias. Let's first think about what bias means from the perspective of $P(T|X)$. If the treatment was randomly assigned with probability, say, 10%, you know that the treatment would not depend on X , or that $P(T|X) = P(T) = 10\%$. So, if the treatment is independent from X , you would not have confounding bias flowing through X and no adjustment would be needed. If you do have this sort of bias, then some units have a higher chance of getting the treatment. For example, it could be that very passionate managers, who already have a very engaged team,

are more likely to take the training (have higher $e(T)$) than the managers whose teams are not so engaged.

If you plot the distribution of $\hat{e}(x)$ by treatment status, since managers don't have the same chance of taking the training (treatment is not random), treated individuals will have higher $\hat{e}(x)$. You can see that in the plot on the left in the following figure, where the treated distribution for $\hat{e}(x)$ is a bit shifted to the right:



Contrast this with the plot on the right. Here, in the low $\hat{e}(X)$ region, treated are up-weighted and control, down-weighted. Similarly, when $\hat{e}(X)$ is high, treated units are down-weighted and control, up-weighted. These movements make the two distributions overlap. The fact that they do means that, on the weighted data, treated and control have the same chance of getting both the treatment or the control. In other words, treatment assignment looks as good as random (assuming no unobserved confounders, of course).

This also sheds some light on what IPW is doing. By taking the treated's outcome, $Y|T = 1$, and up-weighting those where $\hat{e}(X)$ is low and down-weighting those where $\hat{e}(X)$, you are trying to figure out what $Y_1|T = 0$ would look like. A similar argument can be made to show how you are also trying to learn $Y_0|T = 1$ by reweighting the control sample by $1/(1 - P(T = 1))$.

Selection Bias

The example used here is meant to show how propensity score weighting can be used to adjust for common causes, making the treatment similar to the control and vice versa. That is, you saw how to use propensity score weighting as a way to account and control for confounding bias. However, IPW can also be used to adjust for selection issues. In fact, the IPW estimator was initially used in this context, as presented by Horvitz and Thompson in 1952. For this reason, you might see the IPW estimator as the Horvitz-Thompson estimator.

To give an example, suppose you want to know the satisfaction of your customers with your app. So you send out a survey asking them to rate your product on a 1-to-5 scale. Naturally, some customers don't respond. But the issue with this is that it can bias your analysis. If the nonrespondents are mostly unsatisfied customers, the result you'll get back from the survey will be an artificially inflated rate.

To adjust for that you can estimate the probability of responding, R , given customer's covariates (like age, income, app usage, etc.), $P(R = 1|X)$. Then, you can reweight those who responded by $1/\hat{P}(R = 1)$. This will up-weight the respondents that look like the nonrespondents (have low $\hat{P}(R = 1)$). With this, an individual who answered the survey will not only account for himself, but for other individuals like him, creating a pseudo-population that should behave like the original one, but as if everyone responded to the survey.

Sometimes (but hopefully not many), you'll have to face both confounding and selection bias together. In this case, you can use the product of the weights for both selection and confounding. Since this product can be quite small, I recommend stabilizing the confounding bias weights with the marginal probability $P(T = t)$:

$$W = \frac{\hat{P}(T = t)}{\hat{P}(R = 1|X)\hat{P}(T = t|X)}$$

Bias-Variance Trade-Off

As the naive data scientist that I was, when I learned about propensity scores I thought “Oh boy! This is huge! I can transform a causal inference problem into a prediction problem. If I can just predict $e(x)$, I'm golden!” Unfortunately, it is not that simple. In my defense, that is an easy mistake to make. At first glance, it does seem that the better your

estimate of the treatment assignment mechanism, the better your causal estimates will be. But that is simply not the case. Remember when you learned about noise-inducing controls, in [Chapter 4](#)? The same logic applies here. If you have a covariate X_k that is a very good predictor of T , this variable will give you a very accurate model of $e(x)$. But if that same variable does not cause Y , it is not a confounder and it will only increase the variance of your IPW estimate. To see this, think about what would happen if you have a very good model of T . This model would output a very high $\hat{e}(x)$ for all treated units (as it correctly predicts that they are treated) and a very low $\hat{e}(x)$ for all the control units (as it correctly predicts that they are untreated). This would leave you no treated units with low $\hat{e}(x)$ to estimate $Y_1|T = 0$ and no control units with high $\hat{e}(x)$ to estimate $Y_0|T = 1$.

In contrast, think about what would happen if the treatment was randomized. In this case, the predictive power of $\hat{e}(x)$ should be zero! In the managers training example, under randomization, managers with higher $\hat{e}(x)$ would not be more likely to participate in the training than those with lower $\hat{e}(x)$. Still, even with no predictive power, this is the best situation you'll get in terms of estimating the treatment effect.

As you can see, there is also a bias-variance trade-off when it comes to IPW. In general, the more precise the propensity score model, the lower the bias. However, a very precise model for $e(x)$ will generate a very imprecise effect estimate.

This means you have to make your model precise enough to control for the bias, but not too much, or you'll run into variance issues.

TRIMMING

One way to lower the variance of the IPW estimator is to trim the propensity score to be always above a certain number—say, 1%—to avoid weights that are too big—say, above 100. Equivalently, you can directly clip the weights to never be too large. The IPW with clipped weights is no longer unbiased, but it might have lower mean square error if the variance reduction is expressive.

Positivity

The bias–variance trade-off can also be viewed in the light of two causal inference assumptions: conditional independence (unconfoundedness) and positivity. The more precise you make your model for $e(x)$, say, by adding more variables to it, the more you go in the direction of making the CIA hold.

However, you also make positivity less plausible for the reasons you already saw: you'll concentrate the treatment in a low $\hat{e}(x)$ region, far away from the controls and vice versa.

The IPW reconstruction is only possible if you have samples to reweight. If there are no treated samples in the region with low propensity score (high chance of being the control), there is no amount of reweight you can do to reconstruct Y_1 in that region. This is what positivity violations look like in terms of IPW. Also, even if positivity is not entirely violated, but some

units have very small or large propensity scores, IPW will suffer from high variance.

To see this on a more intuitive level, consider the following simulated data. Here, the true ATE is 1. However, x confounds the relationship between T and Y . The higher the X , the smaller the Y , but the higher the chance of receiving the treatment. Hence, a naive comparison in the average outcome between treated and control will be downward biased and can even be negative:

```
In [18]: np.random.seed(1)

n = 1000
x = np.random.normal(0, 1, n)
t = np.random.normal(x, 0.5, n) > 0

y0 = -x
y1 = y0 + t # ate of 1

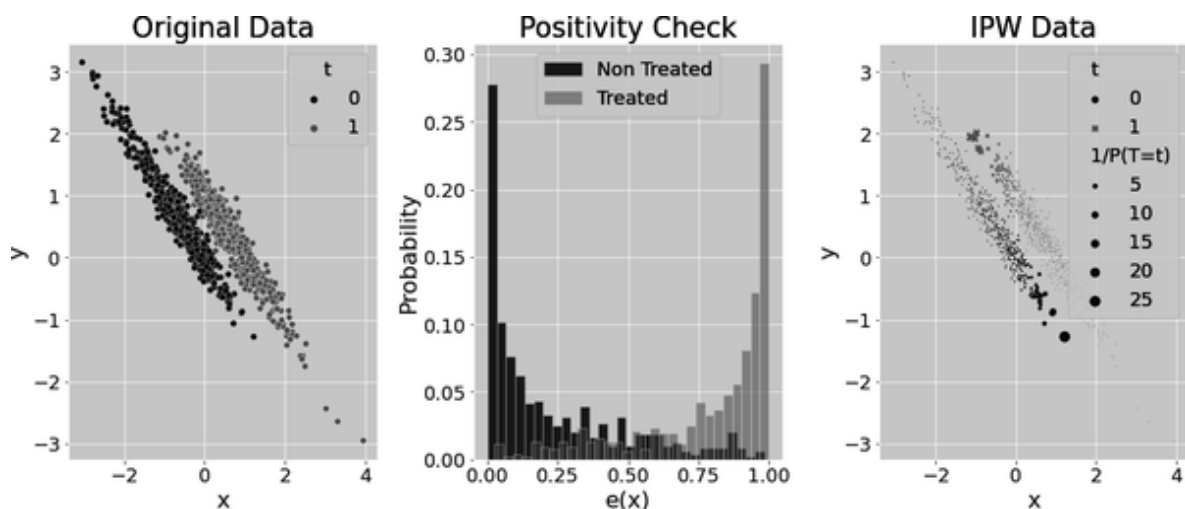
y = np.random.normal((1-t)*y0 + t*y1, 0.5, n)

df_no_pos = pd.DataFrame(dict(x=x, t=t, y=y))

df_no_pos.head()
```

	x	t	y
0	1.624345	1	-0.526442
1	-0.611756	0	0.659516
2	-0.528172	0	0.438549
3	-1.072969	0	0.950810
4	0.865408	1	-0.271397

If you estimate the propensity score in this data, some units (with high x) have a propensity score very close to 1, meaning it is almost impossible for them to get the control. In a similar manner, some units have almost zero chance of getting the treatment (those with low x). You can see this in the following image's middle plot. Notice the lack of overlap between the treated and untreated propensity score distribution. This is very troublesome, as it means that a huge chunk of the control distribution is condensed where $e(x)$ is close to zero, but you have no treated unit to reconstruct that region. As a consequence, you end up lacking a good estimate for $Y_1|T = 0$ for a good portion of the data:



Additionally, as you can see in the third plot, the control units to the right (high $e(x)$) have massive weights. A similar thing can be said about the treated units to the left (small $e(x)$). As you know by now, these huge weights will generally increase the variance of the IPW estimator.

Combine these two problems—high variance and positivity violation—and you’ll see how the IPW estimator fails to recover the ATE of 1 in this data:

```
In [19]: est_fn = partial(est_ate_with_ps, ps_fn=ps_fn)
          print("ATE:", est_fn(df_no_pos))
          print(f"95% C.I.: ", bootstrap(df_no_pos, est_fn))
```

```
Out[19]: ATE: 0.6478011810615735
          95% C.I.: [0.41710504 0.88840195]
```


It's important to note that this isn't simply a problem of high variance. Sure, the 95% CI of this estimator is large, but it is more than that. Specifically, the upper end of the confidence interval still appears to be significantly lower than the true ATE of 1.

Lack of positivity is a problem not only for the IPW estimator. However, IPW can be more transparent about positivity issues. For instance, if you plot the distribution of the propensity score (the plot in the middle of the preceding image) for the treatment variants, you can visually check if you have decent levels of positivity.

In fact, let's contrast the IPW estimator with linear regression. You know that regression will not be very transparent about positivity violations. Instead, it will extrapolate to the regions where you have no data whatsoever. In some very lucky situations, this might even work. For instance, in this very simple simulated data, regression manages to recover the ATE of 1, but only because it correctly extrapolates both Y_0 and Y_1 to the treated and control region where there is no actual data:

```
In [20]: smf.ols("y ~ x + t", data=df_no_pos).f
```

```
Out[20]: 1.0165855487679483
```

In a sense, regression can replace the positivity assumption for a parametric assumption on $E[Y|T, X]$, which is essentially an assumption about smoothness in the potential outcomes. If the linear model has a good fit to the conditional expectation, it will manage to recover the ATE even in regions where positivity doesn't hold. In contrast, IPW makes no assumptions on the shape of the potential outcome. As a result, it fails when extrapolation is needed.

Design- Versus Model-Based Identification

You've just learned how to use propensity score weighting to estimate the treatment effect. Along with regression, this already gives two—and the most important—methods to debias nonexperimental data. But which one should you use and when? Regression or IPW?

Implicit in this choice is the discussion of *model-based versus design-based* identification. Model-based identification involves making assumptions in the form of a model of the potential outcomes conditioned on the treatment and additional covariates. From this perspective, the goal is to

impute the missing potential outcomes required for estimation. In contrast, design-based identification is all about making assumptions about the treatment assignment mechanism. In [Chapter 4](#), you saw how regression fits both kinds of strategy: from the perspective of orthogonalization, it is design-based; from the perspective of an estimator for the potential outcome model, it is model-based. In this chapter, you learned about IPW, which is purely design-based, and in later chapters, you'll learn about Synthetic Control, which is purely model-based.

So, in order to choose between a design- or model-based identification, you need to ask yourself which type of assumption you are more comfortable with. Do you have a good understanding of how the treatment was assigned? Or do you have a better chance in correctly specifying a potential outcome model?

Doubly Robust Estimation

The good news is that, when in doubt, you can just choose both! Doubly robust (DR) estimation is a way of combining both model- and design-based identification, hoping that at least one of them is correct. Here, let's see how to combine propensity score and linear regression in a way that only one of them needs to be rightly specified. Let me show you a popular DR estimator and tell you why it is awesome.

Quite generally, a doubly robust estimator for the counterfactual Y_t can be written as follows:

$$\hat{\mu}_t^{DR}(\hat{m}, \hat{e}) = \frac{1}{N} \sum \hat{m}(X) + \frac{1}{N} \sum \left[\frac{T}{\hat{e}(x)} (Y - \hat{m}(X)) \right]$$

where $\hat{m}(X)$ is a model for $E[Y_t|X]$ (linear regression, for example) and $\hat{e}(X)$ is a propensity score model for $P(T|X)$. Now, the reason why this is amazing—and why it is called doubly robust—is that it only requires one of the models, $\hat{m}(X)$ or $\hat{e}(X)$, to be correctly specified.

For example, suppose that the propensity score model was wrong, but the outcome model $\hat{m}(X)$ was correct. In this case, the second term would converge to zero, since $E[Y - \hat{m}(X)] = 0$. You would be left with the first term, which is just the outcome model, which is correct.

Next, let's consider a scenario where the outcome model is incorrect, but the propensity score model is accurate. To explore this possibility, you can perform some algebraic manipulation on the preceding formula and rewrite it as follows:

$$\hat{\mu}_t^{DR}(\hat{m}, \hat{e}) = \frac{1}{N} \sum \frac{TY}{\hat{e}(X)} + \frac{1}{N} \sum \left[\frac{T - \hat{e}(X)}{\hat{e}(X)} \hat{m}(X) \right]$$

I hope this makes it more clear. If the propensity model is correct, $T - \hat{e}(X)$ would converge to zero. That would leave you only the first term, which is the IPW estimator. And since the propensity model is correct, this estimator would be too.

That's the beauty of this doubly robust estimator: it converges to whichever model is correct.

The preceding estimator would estimate the average counterfactual outcome Y_t . If you want to estimate the average treatment effect, all you have to do is put two of those estimators together, one for $E[Y_0]$ and one for $E[Y_1]$, and take the difference:

$$ATE = \hat{\mu}_1^{DR}(\hat{m}, \hat{e}) - \hat{\mu}_0^{DR}(\hat{m}, \hat{e})$$

Having understood the theory behind DR, it's time to code it up. The models $\hat{e}$ and $\hat{m}$ don't have to be a logistic and linear regression, respectively, but I think those are very good candidates for a starter. Once again, I'll begin by using the R-style formula from `patsy`'s `dmatrix` to engineer my covariate matrix X . Next, I'm using logistic regression to fit the propensity model and get $\hat{e}(X)$. Then comes the output model part. I'm fitting one linear regression per treatment variant, giving me two of them—one for the treated and one for the control. Each model is fitted on the subset of the data of its treatment variant, but makes predictions for the entire dataset. For example, the control model fits only in the data where $T = 0$, but it predicts everything. This prediction is an estimate for Y_0 .

Finally, I'm combining the two models to form the DR estimator for both $E[Y_0]$ and $E[Y_1]$. This is simply the translation of the formula you just saw into code:

```

In [21]: from sklearn.linear_model import LinearRegression

def doubly_robust(df, formula, T, Y):
    X = dmatrix(formula, df)

    ps_model = LogisticRegression(penalty='l2', max_iter=1000)
    ps = ps_model.predict_proba(X)[:, 1]

    m0 = LinearRegression().fit(X[df[T] == 0])
    m1 = LinearRegression().fit(X[df[T] == 1])

    m0_hat = m0.predict(X)
    m1_hat = m1.predict(X)

    return (
        np.mean(df[T] * (df[Y] - m1_hat)),
        np.mean((1 - df[T]) * (df[Y] - m0_hat))
    )

```

Let's see how it performs in the manager training data. You can also pass it to your bootstrap function to construct a confidence interval for the DR ATE estimate:

```

In [22]: formula = """tenure + last_engagement_score +
    + C(n_of_reports) + C(gender) + C(role)
    T = "intervention"
    Y = "engagement_score"

```

```
print("DR ATE:", doubly_robust(df, formula, est_fn))

est_fn = partial(doubly_robust, formula)
print("95% CI", bootstrap(df, est_fn))
```

```
Out[22]: DR ATE: 0.27115831057931455
          95% CI [0.23012681 0.30524944]
```

As you can see, the result is pretty in line with both the IPW and the regression estimator you saw earlier. This is good news, as it means the DR estimator is not doing anything crazy. But honestly, it is kind of boring and it doesn't exactly show the power of DR. So, to better understand why DR is so interesting, let's craft two new examples. They will be fairly simple, but very illustrative.

Treatment Is Easy to Model

The first example is one where the treatment assignment is fairly easy to model, but the outcome model is a bit more complicated. Specifically, the treatment follows a Bernoulli distribution with probability given by the following propensity score:

$$e(x) = \frac{1}{1 + e^{-(1+1.5x)}}$$

In case you didn't recognize, this is exactly the kind of form the logistic regression assumes, so it should be pretty easy to estimate it. Moreover, since $P(T|X)$ is easy to model, the IPW score should have no problem finding the true ATE here, which is close to 2. In contrast, since the outcome Y is a bit trickier, a regression model might run into some trouble:

```
In [23]: np.random.seed(123)

n = 10000
x = np.random.beta(1, 1, n).round(2)*2
e = 1/(1+np.exp(-(1+1.5*x)))
t = np.random.binomial(1, e)

y1 = 1
y0 = 1 - 1*x**3
y = t*(y1) + (1-t)*y0 + np.random.normal(0, 1)

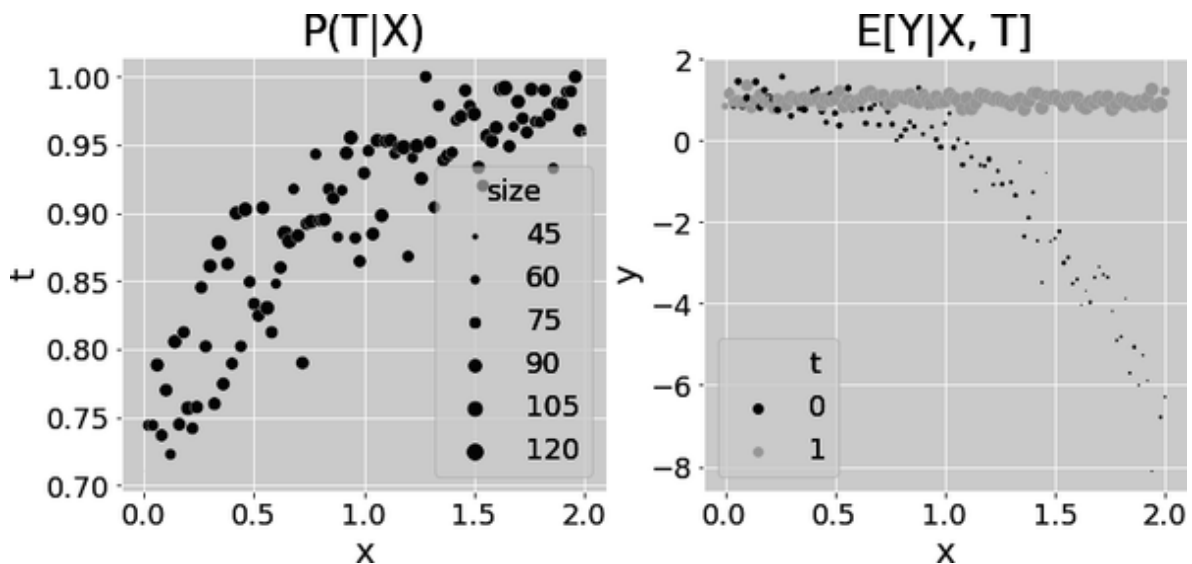
df_easy_t = pd.DataFrame(dict(y=y, x=x, t=t))

print("True ATE:", np.mean(y1-y0))
```

```
Out[23]: True ATE: 2.0056243152
```

The following two plots show what this data looks like. It is interesting to notice the effect heterogeneity in the data, which

is easy to see in the second plot. Notice how the effect is 0 for low values of x and it increases nonlinearly as x increases. This sort of heterogeneity is oftentimes hard for regression to get it right:



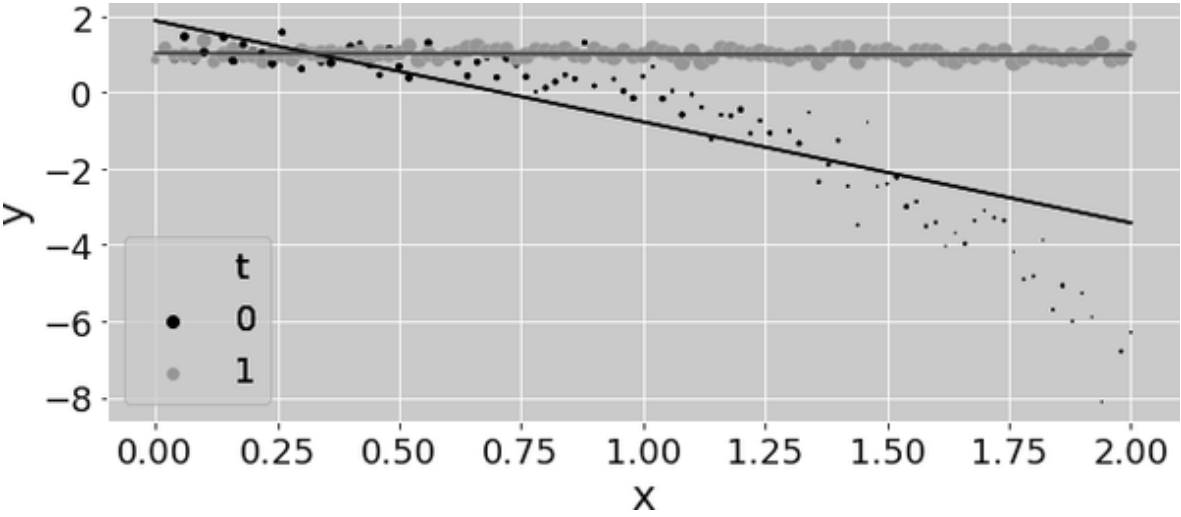
Now, let's see how regression does in this data. Here I'm once again fitting $\hat{m}_1$ and $\hat{m}_0$ separately and estimating the ATE as the average of the different predictions in the entire dataset, $N^{-1} \sum (\hat{m}_1(x) - \hat{m}_0(x))$:

```
In [24]: m0 = smf.ols("y~x", data=df_easy_t.que:
          m1 = smf.ols("y~x", data=df_easy_t.que:
          regr_ate = (m1.predict(df_easy_t) - m0

          print("Regression ATE:", regr_ate)
```

```
Out[24]: Regression ATE: 1.786678396833022
```

As expected, the regression model fails to recover the true ATE of 2. If you plot the predicted values against the original data you can see why. Regression fails to capture the curvature in the control group:



To be clear, this doesn't mean it is not possible to correctly estimate the ATE with regression. If you knew about the true curvature of the data, you could pretty much model it correctly:

```
In [25]: m = smf.ols("y~t*(x + np.power(x, 3))")
         regr_ate = (m.predict(df_easy_t.assign(
                     - m.predict(df_easy_t.assign(
```

```
print("Regression ATE:", regr_ate)
```

```
Out[25]: Regression ATE: 1.9970999747190072
```

But, of course, in reality, you don't really know how the data was generated. So, more likely than not, regression would have failed you here. In contrast, let's see how IPW does. Again, since it is pretty easy to model the treatment assignment, you should expect IPW to perform quite well on this data:

```
In [26]: est_fn = partial(est_ate_with_ps, ps_fn=ps_fn)
print("Propensity Score ATE:", est_fn(0))
print("95% CI", bootstrap(df_easy_t, est_fn, 1000))
```

```
Out[26]: Propensity Score ATE: 2.002350388474011
95% CI [1.80802227 2.22565667]
```

Notice how IPW pretty much nails the correct ATE.

Finally, the moment you've been waiting for, let's see the DR estimate in action. Remember, DR requires one of the models

— $P(T|X)$ or $E[Y_t|X]$ —to be correct, but not necessarily both. In this data, the model for $P(T|X)$ will be correct, but the model for $E[Y_t|X]$ will be wrong:

```
In [27]: est_fn = partial(doubly_robust, formula="Y1 + Y2",
                        print("DR ATE:", est_fn(df_easy_t))
                        print("95% CI", bootstrap(df_easy_t, est_fn))
```

```
Out[27]: DR ATE: 2.001617934263116
          95% CI [1.87088771 2.145382]
```

As expected, the DR performs quite well here, also recovering the true ATE. But there is more. Notice how the 95% CI is smaller than that of pure IPW estimate, meaning the DR estimator is more precise here. This simple example shows how DR can perform well when $P(T|X)$ is easy to model even if it gets $E[Y_t|X]$ wrong. But what about the other way around?

Outcome Is Easy to Model

In this next simple yet illustrative example, the complexity is in $P(T|X)$ rather than $E[Y_t|X]$. Notice the nonlinearity in $P(T|X)$, while the outcome function is simply linear. Here, the true ATE is -1 :

```

In [28]: np.random.seed(123)

n = 10000
x = np.random.beta(1,1, n).round(2)*2
e = 1/(1+np.exp(-(2*x - x**3)))
t = np.random.binomial(1, e)

y1 = x
y0 = y1 + 1 # ate of -1
y = t*(y1) + (1-t)*y0 + np.random.normal(0,1)

df_easy_y = pd.DataFrame(dict(y=y, x=x))

print("True ATE:", np.mean(y1-y0))

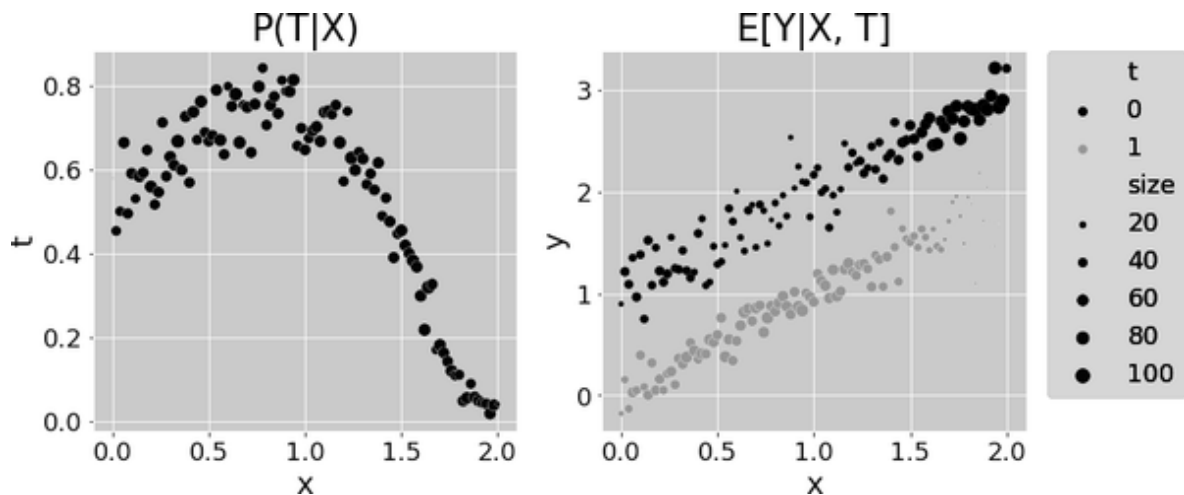
```

```

Out[28]: True ATE: -1.0

```

The same kind of plot from before can be used to show the complex functional form for $P(T|X)$ and the simplicity of $E[Y_t|X]$:



With this data, since the propensity score is relatively complex to model, IPW does not manage to recover the true ATE:

```
In [29]: est_fn = partial(est_ate_with_ps, ps_fn=ps_fn)
         print("Propensity Score ATE:", est_fn(0.5))
         print("95% CI", bootstrap(df_easy_y, est_fn, 1000))
```

```
Out[29]: Propensity Score ATE: -1.10429002786808
         95% CI [-1.14326893 -1.06576358]
```

But regression manages to get it precisely right:

```
In [30]: m0 = smf.ols("y~x", data=df_easy_y.query("t==0"))
         m1 = smf.ols("y~x", data=df_easy_y.query("t==1"))
         regr_ate = (m1.predict(df_easy_y) - m0.predict(df_easy_y)).mean()
```

```
print("Regression ATE:", regr_ate)
```

```
Out[30]: Regression ATE: -1.0008783612504342
```

Once again, because DR only needs one of the models to be correctly specified, it also manages to recover the true ATE here:

```
In [31]: est_fn = partial(doubly_robust, formula1=formula1, formula2=formula2)
print("DR ATE:", est_fn(df_easy_y))
print("95% CI", bootstrap(df_easy_y, est_fn))
```

```
Out[31]: DR ATE: -1.0028459347805823
          95% CI [-1.04156952 -0.96353366]
```

I hope these two examples made it more clear why doubly robust estimation can be very interesting. The bottom line is that it gives you two shots at being correct. In some cases, it's hard to model $P(T|X)$, but easy to model $E[Y_t|X]$. In others, the reverse might be true. Regardless, as long as you can model one of them correctly, you can combine a model for $P(T|X)$ and a model for $E[Y_t|X]$ in a way that only one of

them needs to be correct. This is the true power of the doubly robust estimator.

SEE ALSO

The DR estimator covered here is only one of the many out there. Just to give some examples, you could take the DR estimator covered in this chapter but fit the regression model with weights set to $\hat{e}(x)$. Or, you could add $\hat{e}(x)$ to the regression model. Interestingly, linear regression alone is a DR estimator that models the treatment as $e(x) = \beta X$. It is not a very good DR estimator, since βX is not bounded between 0 and 1, as a probability model should be, but it is nonetheless a DR estimator. To learn more about other DR estimators, check out the excellent discussion in “Comment: Performance of Double-Robust Estimators When ‘Inverse Probability’ Weights Are Highly Variable,” 2008, by Robins et al.

Generalized Propensity Score for Continuous Treatment

Until now, this chapter has only shown how to use propensity scores for discrete treatment. There is a pretty good reason for that. Continuous treatments are way more complicated to deal with. So much so that I would say that causal inference as a science doesn’t have a very good answer on how to deal with them.

In [Chapter 4](#), you managed to get away with continuous treatment by assuming a functional form for the treatment response. Something like $y = a + bt$ (linear form) or $y = a + b\sqrt{t}$ (square root form), which you could then estimate with OLS. But when it comes to propensity weighting,

there is no such thing as a parametric response function. The potential outcomes are estimated nonparametrically, by reweighting and taking averages. When T is continuous, there exist infinitely many potential outcomes Y_t . Furthermore, since the probability of a continuous variable is always zero, it is not feasible to estimate $P(T = t|X)$ in this scenario.

The simplest way out of these issues is to discretize the continuous treatment into a coarser version that can then be treated as discrete. But there is another way out, which is to use the generalized propensity score. If you make some changes to the traditional propensity score, you'll be able to accommodate any type of treatment. To see how this would work, consider the following example.

A bank wants to know how a loan's interest rates affect the duration (in months) that the customer chooses to pay back that loan. Intuitively speaking, the effect of interest on the duration should be negative, since people like to pay back high-rate loans as fast as possible to avoid paying too much on interest.

To answer this question, the bank could randomize the interest rate, but this would be costly, both in money and in time. Instead, it wants to use the data it already has. The bank knows that interest rates were assigned by two machine learning models: `m1_1` and `m1_2`. Additionally, since the bank's data scientists were very smart, they added a random Gaussian noise to the interest rate decision-making

process. This ensures that the policy is nondeterministic and that the positivity assumption is not violated. The observational (nonrandomized) interest data, along with information on the confounders `ml_1` and `ml_2` and the outcome `duration` is stored in the `df_cont_t` data frame:

```
In [32]: df_cont_t = pd.read_csv("./data/interests.csv")
df_cont_t.head()
```

	ml_1	ml_2	interest
0	0.392938	0.326285	7.1
1	-0.427721	0.679573	5.6
2	-0.546297	0.647309	11.1
3	0.102630	-0.264776	7.2
4	0.438938	-0.648818	9.5

Your task is to unbiased the relationship between interest rate and duration, adjusting for `m1_1` and `m1_2`. Notice that, if you estimate the treatment effect naively, not adjusting for anything, you'll find a positive treatment effect. As discussed already, this makes no business sense, so this result is probably biased:

```
In [33]: m_naive = smf.ols("duration ~ interest")
         m_naive.summary().tables[1]
```

	coef	std err	t	P> t
Intercept	14.5033	0.226	64.283	0.000
interest	0.3393	0.029	11.697	0.000

To adjust for `m1_1` and `m1_2`, you could just include them in your model, but let's see how to manage the same thing with reweighting. The first challenge that needs to be addressed is the fact that continuous variables have a probability of zero everywhere, i.e., $P(T = t) = 0$. This occurs because the probability is represented by the area under the density, and the area of a single point is always zero. One possible solution is to work with the conditional density function $f(T|X)$ instead of the conditional probability

$P(T = t|X)$. However, this approach presents another issue, which is specifying the distribution of the treatment.

Here, for simplicity's sake, let's assume it is drawn from a normal distribution $T \sim N(\mu_i, \sigma^2)$. This is a fairly reasonable simplification, especially since the normal distribution can be used to approximate other distributions. Moreover, let's assume constant variance σ^2 , instead of one that changes for each individual.

Recall that the density of the normal distribution is given by the following function:

$$f(t_i) = \frac{\exp\left(-\frac{1}{2}\left(\frac{t_i - \mu_i}{\sigma}\right)^2\right)}{\sigma\sqrt{2\pi}}$$

Now you need to estimate the parameters of this conditional Gaussian, that is, the mean and standard deviation. The simplest way to do that is using OLS to fit the treatment variable:

```
In [34]: model_t = smf.ols("interest~ml_1+ml_2",
```

Then, the fitted values will be used as μ_i and the standard deviation of the residual will be σ . With this, you have an estimate for the conditional density. Next, you'll need to evaluate that conditional density at the given treatment,

which is why I'm passing T to the `x` argument in the density function in the following code:

```
In [35]: def conditional_density(x, mean, std):  
        denom = std*np.sqrt(2*np.pi)  
        num = np.exp(-((1/2)*((x-mean)/std)  
        return (num/denom).ravel()  
  
        gps = conditional_density(df_cont_t["in  
                                model_t.fitted  
                                np.std(model_  
  
        gps
```

```
Out[35]: array([0.1989118, 0.14524168, 0.033384  
              0.15732008])
```

Alternatively, you can (and probably should) import the normal distribution from `scipy` and use that instead:

```
In [36]: from scipy.stats import norm  
  
        gps = norm(loc=model_t.fittedvalues,  
                   scale=np.std(model_t.resid))  
  
        gps
```

```
Out[36]: array([0.1989118, 0.14524168, 0.0333842,
               0.15732008])
```

BEYOND THE NORMAL

If the treatment follows another distribution other than the normal, you can use generalized linear models (`glm`) to fit it. For example, if T was assigned according to a Poisson distribution, you could build the GPS weights with something like the following code:

```
import statsmodels.api as sm
from scipy.stats import poisson

mt = smf.glm("t~x1+x2",
             data=df, family=sm.families.Poisson()).fit()

gps = poisson(mu=m_pois.fittedvalues).pmf(df["t"])

w = 1/gps
```

Using the inverse of the GPS as weights in a regression model can adjust for the bias. You can see that now you'll find a negative effect of interest on duration, which makes more business sense:

```
In [37]: final_model = smf.wls("duration~interest",
                               data=df_cont_t, w=w)
```

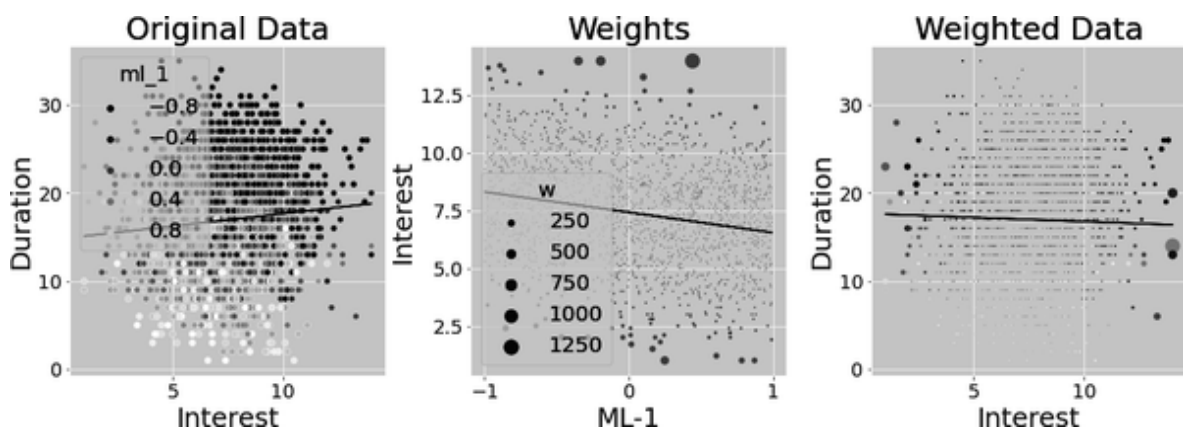
```
final_model.params["interest"]
```

```
Out[37]: -0.6673977919925854
```

There is still one more improvement that can be made, which will provide a more intuitive understanding of GPS. Using this score to construct weights will assign greater importance to points with unlikely treatments. Specifically, you will assign high weights to units with high residuals in the treatment model that you have fitted. Additionally, due to the exponential nature of the normal density, the weights will increase exponentially with the size of the residual.

To illustrate this point, suppose you fit the interest rate using only `ml_1`, instead of both `ml_1` and `ml_2`. This simplification enables presenting everything in a single plot. The resulting weights are displayed in the next figure. The first plot shows the original data, color-coded by the confounder `ml_1`. Customers with low scores on `ml_1` typically select longer durations to repay their loan. Furthermore, customers with low `ml_1` scores are assigned higher interest rates. Consequently, an upward bias exists in the relationship between interest rate and duration.

The second plot shows the fitted values of the treatment model and the weights constructed by the GPS, obtained from that model. They are larger the farther you go from the fitted line. This makes sense, as the GPS gives more weight to unlikely treatments. But look how big the weights can get. Some are bigger than 1,000!



The last plot shows the same weights, but in the relationship between interest and duration. Since both low interest rates at low values of `ml_1` and high interest at high values of `ml_1` are unlikely, inverse GPS weight gives high importance to those points. This manages to reverse the positive (and biased) relationship between interest and duration. But this estimator will have huge variance, as it is practically just using a few data points—those with very high weights. Moreover, because this data was simulated, I know for a fact that the true ATE is -0.8 , but the preceding estimate is only -0.66 .

To improve upon it, you can stabilize the weights by the marginal density $f(t)$. Unlike with discrete treatment, where weight stabilization was just nice to have, with the GPS, I would say it is a must. To estimate $f(t)$, you can simply use the average treatment value. Then, evaluate the resulting density at the given treatments.

Notice how this produces weights that sum to (almost) the original sample size. Thinking about this in the light of importance sampling, these weights take you from $f(t|x)$ to $f(t)$, a density where the treatment does not depend on x :

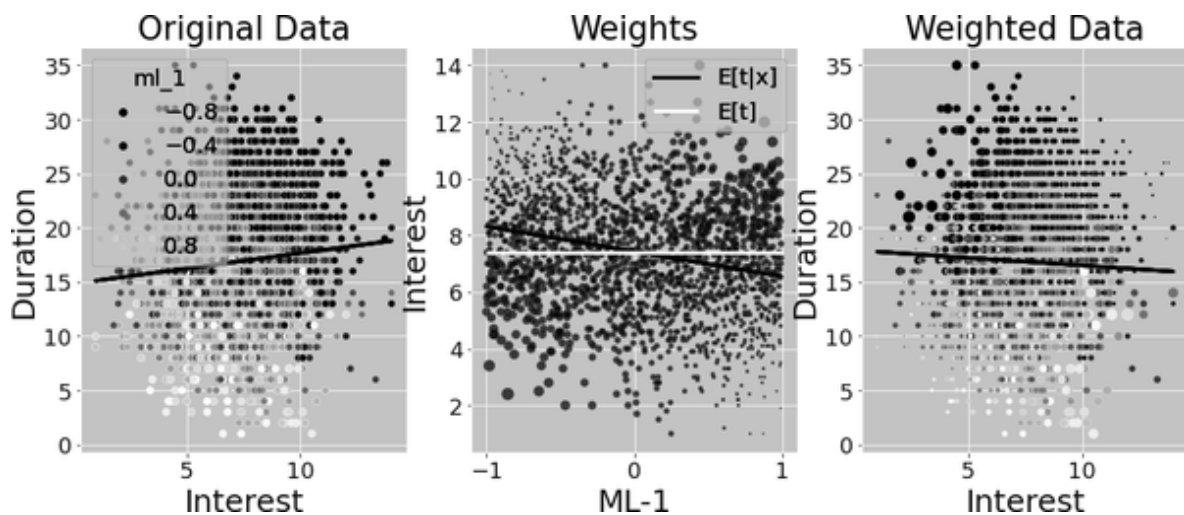
```
In [38]: stabilizer = norm(
          loc=df_cont_t["interest"].mean(),
          scale=np.std(df_cont_t["interest"]
          ).pdf(df_cont_t["interest"])

          gipw = stabilizer/gps

          print("Original Sample Size:", len(df_c
          print("Effective Stable Weights Sample
```

```
Out[38]: Original Sample Size: 10000
          Effective Stable Weights Sample Size: 9999
```

Again, to understand what is going on, suppose you fit $f(t|x)$ using only `ml_1`. Once again, inverse propensity weighting gives high importance to points that are far from the fitted values of the treatment model, as they fall in a low-density region of $f(t|x)$. But additionally, the stabilization also gives low importance to points that are far away from $f(t)$, that is, points far from the mean. The result is twofold. First, the stabilized weights are much smaller, which gives you lower variance. Second, it becomes clearer that you are now giving more importance to points with both low values of `ml_1` and low interest rate (and vice versa). You can see this by the change in the color pattern between the first and third plots:



Also, these stabilized weights give you an estimate that is much closer to the true ATE of -0.8 :

```
In [39]: final_model = smf.wls("duration ~ interest",
                                data=df_cont_t, weights=weights)

final_model.params["interest"]
```

```
Out[39]: -0.7787046278134069
```

As you can see, even though weight stabilization didn't have an impact in the case where T was discrete, it is very relevant for continuous treatments. It gets you closer to the true value of the parameter you are trying to estimate and it also significantly reduces the variance. Since it is a bit repetitive, I'm omitting the code to compute the 95% CI of the estimates, but it is pretty much what you did before: just wrap the whole thing in a function and bootstrap it. But just so you can see it for yourself, here are the 95% CI with and without stabilization:

```
95% CI, non-stable:  [-0.81074164 -0.52605933]
95% CI, stable:      [-0.85834311 -0.71001914]
```

Notice how both contain the true value of -0.8 , but the one with stabilization is much narrower.

CONTINUOUS TREATMENT LITERATURE

There are other ways to estimate the treatment effect with models that predict the treatment. One idea (by Hirano and Imbens) is to include the GPS as a covariate in a regression function. Another option (by Imai and van Dyk) is fit T , segment the data based on the predictions $\hat{T}$, regress the treatment on the outcome on each segment defined by $\hat{T}$, and combine the results using a weighted average, where the weights are the size of each group.

For a more comprehensive survey of the available options, I recommend checking out Douglas Galagate's PhD thesis, "Causal Inference with a Continuous Treatment and Outcome."

There is also a Python package named `causal-curve` that provides a scikit-learn-like API for modeling continuous treatment with GPS, if you don't want to code all of this by hand.

Key Ideas

Along with regression—and orthogonalization in general—inverse propensity weighting is the second workhorse for bias adjustment in your causal inference toolbox. Both techniques require you to model the treatment. This should serve as a reminder of how important it is to think about the treatment

assignment mechanism in any causal inference problem. However, each technique makes use of that treatment model in a very unique way. Orthogonalization residualized the treatment, projecting it to a new space where it becomes linearly independent (orthogonal) to the covariates X that were used to model the treatment. IPW keeps the same treatment dimension, but reweights the data by the inverse of the treatment propensity:

$$w = \frac{P(T)}{P(T|X)}$$

This makes it look like the treatment was drawn from a distribution $P(T)$, which does not depend on the covariates X that were used to create the propensity model.

[Figure 5-1](#) shows a simple comparison between the two approaches. In this data, treatment effect is positive, but confounded by x , which is depicted in the color schema of the data points. The first plot contains the original data along a regression line of y on y . The negative slope is due to the bias that comes from x . The next two plots show how orthogonalization and IPW debias this data using very distinct ideas. Both manage to recover a positive causal effect of t on y , as shown by their respective regression lines.

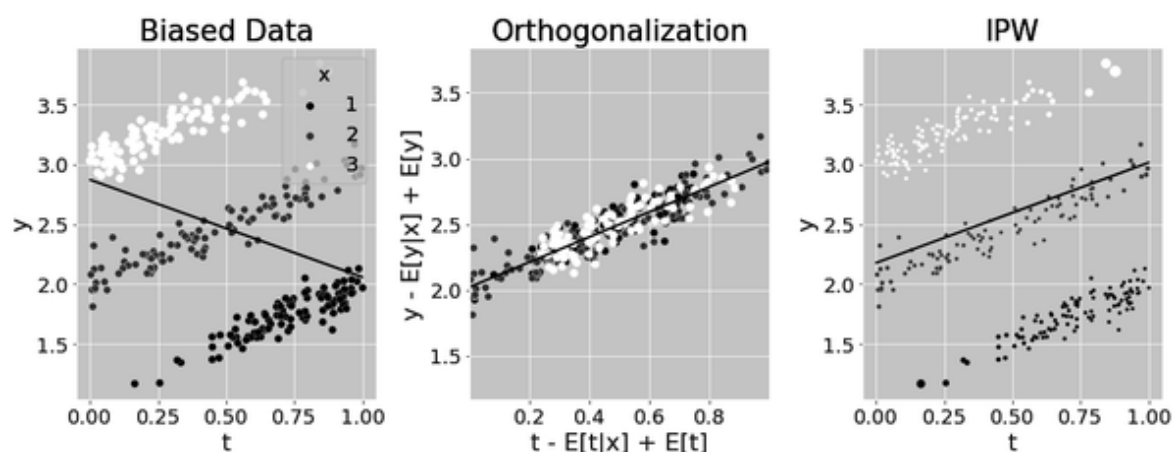


Figure 5-1. How orthogonalization and IPW remove bias

If both procedures manage to debias the data, a natural question that arises is which one should you choose. This is a bit personal, but here is my take on it. I really like IPW for when the treatment is discrete, especially if you pair it with outcome modeling in a doubly robust approach. However, when the treatment is continuous, I tend to gravitate toward regression modeling of the kind you saw in [Chapter 4](#). With continuous treatment, you'll have very few data points around any specific treatment. As a result, a method like IPW, which doesn't pose a parametric assumption on the treatment response function, becomes less appealing. For me, it is more productive to assume some smoothness in the treatment response function, allowing you to pool information from neighboring points around a particular treatment to infer its response.

Sure, I think it is very much worth understanding approaches like the Generalized Propensity Score, as it gives you further intuition into IPW in general. That is why I've included it in

this chapter. Also, as the continuous treatment literature advances, I want you to be able to keep up with it, if you wish to. But, day-to-day, when T is continuous, I think you'll be better off with outcome models like linear regression.